



**JAX: Functional
Transformations for
High-Performance
ML (2018) -
developed by
Google**

- Transform numerical Python/NumPy functions
- Functional programming paradigm; immutable arrays
- XLA (accelerated linear algebra)-backed just-in-time compilation for CPUs/GPUs/TPUs

Why It Matters:

- Research velocity with concise, pure functions
- Performance: kernel fusion, layout optimization, device placement via XLA
- Scales from single device to multi-accelerator setups (TPUs/GPUs)

Just-in-Time (JIT) compilation is a method for executing computer code that translates it into native machine code at runtime. It is a hybrid approach that combines the best aspects of traditional interpreters (flexibility) and static, ahead-of-time (AOT) compilers (performance).

XLA, which stands for Accelerated Linear Algebra, is an open-source compiler infrastructure designed to improve the performance of machine learning models and other numerical computations, particularly those involving large-scale array operations.

Framework Comparison Table

	TensorFlow	PyTorch	JAX
Graph Type	Static (1.x), Dynamic (2.x)	Dynamic	Functional transformations
Programming Model	Imperative (2.x), Symbolic (1.x)	Imperative	Functional
Core Data Structure	Tensor (mutable)	Tensor (mutable)	Array (immutable)
Execution Mode	Eager (2.x default), Graph	Eager	Just-in-time compilation
Automatic Differentiation	Reverse mode	Reverse mode	Forward and Reverse mode
Hardware Acceleration	CPU, GPU, TPU	CPU, GPU	CPU, GPU, TPU

Eager execution in TensorFlow is an imperative programming environment where operations are evaluated immediately as they are called, rather than building a computational graph to be run later. This contrasts with TensorFlow's traditional graph mode, where operations are first added to a symbolic graph and then executed in a TensorFlow session.

Week 2 Overview

Automatic Differentiation: The Engine of Modern ML

Learning Objectives:

- Master forward and reverse mode automatic differentiation algorithms
- Analyze memory and computational trade-offs between AD modes
- Evaluate AD implementation strategies for resource-constrained environments
- Design efficient gradient computation systems for edge and embedded deployment

Required Readings:

- Book 1 - Volume 1: Neural Computation
- Book 1 - Volume 1: ML Frameworks
- Book 1 - Volume 1: Model Training
- Book 2: Chapter 3: Getting Up to Speed on Machine Learning



NETFLIX



Automated Compilation Pipelines - XLA

```
import tensorflow as tf

# Simple model component
class Block(tf.Module):
    def __init__(self, c): super().__init__(); self.conv =
tf.keras.layers.Conv2D(c, 3, padding="same")
    def __call__(self, x, training=False):
        x = self.conv(x)
        x = tf.keras.layers.BatchNormalization()(x,
training=training)
        return tf.nn.relu(x)

# Enable XLA for this function (TF >= 2.11: jit_compile=True)
@tf.function(jit_compile=True)
def step(x):
    b = Block(32)
    return b(x, training=False)

# Warm up + run
x = tf.random.normal([1, 224, 224, 3])
y = step(x) # XLA compiles on first call; later calls use
optimized code
```

What XLA (Accelerated Linear Algebra) does

- Fuses ops into single kernels
- Picks efficient layouts (e.g., NHWC ↔ NCHW)
- Reuses buffers to cut memory

When to use

- TensorFlow models with stable shapes
- Training or inference on CPU/GPU/TPU

When to be careful

- Highly dynamic Python control flow
- Ragged tensors or frequent shape changes

How to enable

- `@tf.function(jit_compile=True)` in TF 2.11+ (or `experimental_compile=True` in older TF)
- Or set `TF_XLA_FLAGS / tf.config.optimizer.set_jit(True)`



Hardware-Specific Optimizations - TPU

TPU Optimizations

Objective: Maximize throughput on Google's TPUs, which are specialized for matrix operations.

Key Considerations:

- TPUs perform best when matrix dimensions are aligned to specific sizes (e.g., multiples of 128).
- Misaligned dimensions can lead to underutilized hardware or wasted computation.

```
# Ensure matrix dimensions are multiples of 128
def tpu_optimize(model):
    for layer in model.layers:
        if isinstance(layer, Dense):
            # Pad dimensions for optimal TPU utilization
            layer.units = align_to_tpu_constraints(layer.units)
```

- **align_to_tpu_constraints:** A helper function that rounds up the number of units (neurons) in a dense layer to the nearest multiple of 128.
- **Why?** TPUs process matrix operations in tiles (e.g., 128x128 blocks). Aligning dimensions ensures full utilization of these tiles, avoiding inefficient partial computations.